

1

Programs

bei Def. von Programmen nutzen wir $x_0, k_1, \dots$
 $\hookrightarrow l_p$ wird erst bei Makroaufruf in einem Programm relevant
 $\Rightarrow x_{ip+1} \hookrightarrow x_0, x_{ip+2} \hookrightarrow k_1, x_{ip+3} \hookrightarrow k_2$

Addition

P add: $(x_{ip+2} + x_{ip+3})$
 $x_{ip+1} := x_{ip+2};$
loop x_{ip+3} **do**
 $x_{ip+1} := \text{suc}(x_{ip+1})$
end
 // cleanup

mul, addel können direkt als Makros genutzt werden, wurden in VL / Übung definiert

$\hookrightarrow P \text{ exp}(k_1 k_2);$
 $x_0 := 1; \text{ // } 0 - x_0 := \text{suc}(x_0)$
loop k_2 **do**
 $x_0 := x_1 \cdot x_0$
end

würde reichen und $\delta((0, 2, 3, 0, \dots), \text{exp})$
 reicht für Abarbeitung

Multiplication

P mult: $(x_{ip+2} \cdot x_{ip+3})$
 $(x_{ip+1} := 0;)$
loop x_{ip+3} **do**
 $x_{ip+1} := x_{ip+1} + x_{ip+2}$
end
 // cleanup

① Programme(n) funktionieren, l_p hier nicht sinnvoll (s.o.), Abarbeitung mit $\delta \Rightarrow$ wäre 30% + 1 BP ü

Exponentiation

P exp: $(x_{ip+2}^{x_{ip+3}})$
 $(x_{ip+1} := 0;)$
 $x_{ip+1} := \text{suc}(x_{ip+1});$ ✓
loop x_{ip+3} **do**
 $x_{ip+1} := x_{ip+1} \cdot x_{ip+2}$ ✓
end
 // cleanup

Evaluation Traces

Addition

passieren implizit in exp

$\delta((0, 3, 2, 0, \dots), P \text{ add}) =$
 $\delta((0, 3, 2, 0, \dots), x_{ip+1} := x_{ip+2}; \text{loop } x_{ip+3} \text{ do } x_{ip+1} := \text{suc}(x_{ip+1}) \text{ end}) =$
 $\delta(\delta((0, 3, 2, 0, \dots), x_{ip+1} := x_{ip+2}), \text{loop } x_{ip+3} \text{ do } x_{ip+1} := \text{suc}(x_{ip+1}) \text{ end}) =$
 $\delta((3, 3, 2, 0, \dots), \text{loop } x_{ip+3} \text{ do } x_{ip+1} := \text{suc}(x_{ip+1}) \text{ end}) =$
 $\delta((3, 3, 2, 0, \dots), x_{ip+1} := \text{suc}(x_{ip+1}); x_{ip+1} := \text{suc}(x_{ip+1})) =$
 $\delta(\delta((3, 3, 2, 0, \dots), x_{ip+1} := \text{suc}(x_{ip+1})), x_{ip+1} := \text{suc}(x_{ip+1})) =$
 $\delta((4, 3, 2, 0, \dots), x_{ip+1} := \text{suc}(x_{ip+1})) =$
 $(5, 3, 2, 0, \dots)$

Multiplication

$$\begin{aligned}
& \delta((0, 3, 2, 0, \dots), P \text{ mult}) = \\
& \delta((0, 3, 2, 0, \dots), x_{ip+1} := 0; \text{loop } x_{ip+3} \text{ do } x_{ip+1} := x_{ip+1} + x_{ip+2} \text{ end}) = \\
& \delta(\delta((0, 3, 2, 0, \dots), x_{ip+1} := 0), \text{loop } x_{ip+3} \text{ do } x_{ip+1} := x_{ip+1} + x_{ip+2} \text{ end}) = \\
& \delta((0, 3, 2, 0, \dots), \text{loop } x_{ip+3} \text{ do } x_{ip+1} := x_{ip+1} + x_{ip+2} \text{ end}) = \\
& \delta((0, 3, 2, 0, \dots), x_{ip+1} := x_{ip+1} + x_{ip+2}; x_{ip+1} := x_{ip+1} + x_{ip+2}) = \\
& \delta(\delta((0, 3, 2, 0, \dots), x_{ip+1} := x_{ip+1} + x_{ip+2}), x_{ip+1} := x_{ip+1} + x_{ip+2}) = \\
& \delta((3, 3, 2, 0, \dots), x_{ip+1} := x_{ip+1} + x_{ip+2}) = \\
& (6, 3, 2, 0, \dots)
\end{aligned}$$

Exp

$$\begin{aligned}
& \delta((0, 2, 3, 0, \dots), P \text{ exp}) = \\
& \delta((0, 2, 3, 0, \dots), x_{ip+1} := 0; x_{ip+1} := \text{suc}(x_{ip+1}); \text{loop } x_{ip+3} \text{ do } x_{ip+1} := x_{ip+1} \cdot x_{ip+2} \text{ end}) = \\
& \delta(\delta((0, 2, 3, 0, \dots), x_{ip+1} := 0), x_{ip+1} := \text{suc}(x_{ip+1}); \text{loop } x_{ip+3} \text{ do } x_{ip+1} := x_{ip+1} \cdot x_{ip+2} \text{ end}) = \\
& \delta((0, 2, 3, 0, \dots), x_{ip+1} := \text{suc}(x_{ip+1}); \text{loop } x_{ip+3} \text{ do } x_{ip+1} := x_{ip+1} \cdot x_{ip+2} \text{ end}) = \\
& \delta(\delta((0, 2, 3, 0, \dots), x_{ip+1} := \text{suc}(x_{ip+1})), \text{loop } x_{ip+3} \text{ do } x_{ip+1} := x_{ip+1} \cdot x_{ip+2} \text{ end}) = \\
& \delta((1, 2, 3, 0, \dots), \text{loop } x_{ip+3} \text{ do } x_{ip+1} := x_{ip+1} \cdot x_{ip+2} \text{ end}) = \\
& \delta((1, 2, 3, 0, \dots), x_{ip+1} := x_{ip+1} \cdot x_{ip+2}; x_{ip+1} := x_{ip+1} \cdot x_{ip+2}; x_{ip+1} := x_{ip+1} \cdot x_{ip+2}) = \\
& \delta(\delta((1, 2, 3, 0, \dots), x_{ip+1} := x_{ip+1} \cdot x_{ip+2}), x_{ip+1} := x_{ip+1} \cdot x_{ip+2}; x_{ip+1} := x_{ip+1} \cdot x_{ip+2}) = \\
& \delta((2, 2, 3, 0, \dots), x_{ip+1} := x_{ip+1} \cdot x_{ip+2}; x_{ip+1} := x_{ip+1} \cdot x_{ip+2}) = \\
& \delta(\delta((2, 2, 3, 0, \dots), x_{ip+1} := x_{ip+1} \cdot x_{ip+2}), x_{ip+1} := x_{ip+1} \cdot x_{ip+2}) = \\
& \delta((4, 2, 3, 0, \dots), x_{ip+1} := x_{ip+1} \cdot x_{ip+2}) = \\
& (8, 2, 3, 0, \dots)
\end{aligned}$$

2

1.

P add: ($x_{ip+2} + x_{ip+3}$)

M_1 : $x_{ip+1} := x_{ip+2}$;

M_2 : **if** x_{ip+3} **goto** M_4 ;

M_3 : **halt**;

M_4 : $x_{ip+1} := \text{suc}(x_{ip+1})$;

M_5 : $x_{ip+3} := \text{pred}(x_{ip+3})$;

M_6 : **if** x_{ip+1} **goto** M_2 ;

P fib: (x_{ip+2} . Fibonacci Zahl)

M_1 : $x_{ip+1} := \text{suc}(0)$;

M_2 : $x_{ip+3} := \text{suc}(0)$;

M_3 : $x_{ip+2} := \text{pred}(\text{pred}(x_{ip+2}))$;

M_4 : **if** x_{ip+2} **goto** M_6 ;

M_5 : **halt**;

M_6 : $x_{ip+4} := x_{ip+1}$;

M_7 : $x_{ip+1} := x_{ip+1} + x_{ip+3}$;

M_8 : $x_{ip+3} := x_{ip+4}$;

M_9 : $x_{ip+2} := \text{pred}(x_{ip+2})$;

M_{10} : **if** x_{ip+1} **goto** M_4 ;

2.

P mult: $(x_{ip+2} \cdot x_{ip+3})$

$M_1 :$ $x_{ip+1} := x_{ip+2};$

$M_2 :$ **if** x_{ip+3} **goto** $M_4;$

$M_3 :$ **halt;**

$M_4 :$ $x_{ip+1} := x_{ip+1} + x_{ip+3};$

$M_5 :$ $x_{ip+3} := \text{pred}(x_{ip+3});$

$M_6 :$ **if** x_{ip+1} **goto** $M_2;$

P fac: $(x_{ip+2}!)$

$M_1 :$ $x_{ip+1} := \text{suc}(0);$

$M_3 :$ **if** $\text{pred}(x_{ip+2})$ **goto** $M_5;$

$M_4 :$ **if** x_{ip+1} **goto** $M_{10};$

$M_5 :$ **if** x_{ip+2} **goto** $M_7;$

$M_6 :$ **halt;**

$M_7 :$ $x_{ip+1} := x_{ip+1} \cdot x_{ip+2};$

$M_8 :$ $x_{ip+2} := \text{pred}(x_{ip+2});$

$M_9 :$ **if** x_{ip+1} **goto** $M_5;$

$M_{10} :$ **if** x_{ip+1} **goto** $M_{10};$

3

Fibonacci

Das GOTO-Programm

P_{fib} : $(x_{ip+2} \cdot \text{Fibonacci-Zahl})$

$P_{\text{setup}};$

if x_{ip+2} **goto** ...;

$P_{\text{rep. add}};$

$x_{ip+2} := \text{pred}(x_{ip+2});$

if x_{ip+1} **goto** ...;

lässt sich durch das folgende WHILE-Programm darstellen:

P_{fib} : $(x_{ip+2} \cdot \text{Fibonacci-Zahl})$

$P_{\text{setup}};$

while $x_{ip+2} \neq 0$ **do**

$P_{\text{rep. add}};$

$x_{ip+2} := \text{pred}(x_{ip+2});$

end

Die einzelnen Programmblöcke bleiben unverändert. Die Äquivalenz beider Programme ergibt sich daraus, dass die Schleifenbedingung direkt übernommen wird und die Modifikation der Schleifenvariablen in beiden Fällen identisch erfolgt.

Factorial

```
P fac: ( $x_{ip+2}!$ )  
   $x_{ip+1} := \text{suc}(0);$   
  while  $\text{pred}(x_{ip+2}) == 0$  do  
    end  
  while  $x_{ip+2}$  do  
     $x_{ip+1} := x_{ip+1} \cdot x_{ip+2};$   
     $x_{ip+2} := \text{pred}(x_{ip+2});$   
  end
```

Die Fakultäts-Funktion wurde auf die selbe Version konvertiert. Die rechnenden Code Blöcke sind identisch geblieben, sowie die Terminations Condition.